

Xử lý ngôn ngữ tự nhiên

Trương Xuân Nam

12 vấn đề cơ bản

12 bài toán cơ bản của NLP (1/2)

1 Phân tách từ (Tokenization)

Chia văn bản thành các đơn vị nhỏ như từ, subword hoặc ký tự để máy xử lý.

Ví dụ: "Tôi yêu Việt Nam!"

↓

[Tôi] [yêu] [Việt Nam] [!]

2 Gán nhãn từ loại (Part-of-Speech Tagging)

Xác định loại từ của từng từ trong câu như danh từ, động từ, tính từ.

Ví dụ: Học sinh chăm chỉ học bài.

Học sinh	chăm chỉ	học	bài
↓	↓	↓	↓
N	Adj	V	N

3 Nhận dạng thực thể (Named Entity Recognition - NER)

Phát hiện tên người, tổ chức, địa điểm, thời gian... trong văn bản.

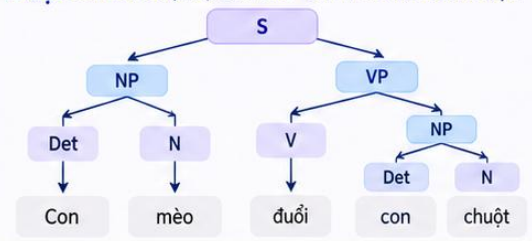
Ví dụ: Nguyễn Văn A làm việc tại Đại học Thủy lợi ở Hà Nội.

PERSON	ORG	LOC
Nguyễn Văn A	Đại học Thủy lợi	Hà Nội

4 Phân tích cú pháp (Syntactic Parsing)

Xác định cấu trúc ngữ pháp và quan hệ giữa các thành phần trong câu.

Ví dụ: Cấu trúc cú pháp của câu "Con mèo đuổi con chuột."



5 Phân tích ngữ nghĩa (Semantic Understanding)

Hiểu vai trò và ý nghĩa của các thành phần trong câu.

Ví dụ: Lan tặng một cuốn sách cho Minh.

Agent (người thực hiện)	Action (hành động)	Theme (đối tượng bị tác động)	Recipient (người nhận)
↓	↓	↓	↓
Lan	tặng	cuốn sách	Minh

6 Giải quyết đồng tham chiếu (Coreference Resolution)

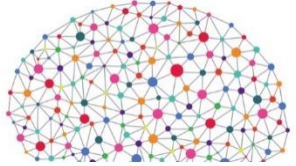
Xác định các đại từ hoặc cụm từ cùng chỉ một đối tượng.

Ví dụ: Nam đến trường. Anh ấy rất chăm chỉ.

Nam ↔ Anh ấy

(cùng chỉ một người)

12 vấn đề cơ bản



12 bài toán cơ bản của NLP (2/2)

7 Phân loại văn bản (Text Classification)

Gán toàn bộ văn bản vào một hoặc nhiều nhãn như chủ đề, thể loại, spam/not spam.

Ví dụ: "Email này trúng thưởng lớn, bấm vào link ngay!" → **Spam** | Không spam

8 Phân tích cảm xúc (Sentiment Analysis)

Xác định cảm xúc hoặc thái độ thể hiện trong câu: tích cực, tiêu cực, trung tính.

Ví dụ: "Bộ phim này rất hay và cảm động." → **Tích cực (+)** | Tiêu cực (-) | Trung tính (0)

9 Dịch máy (Machine Translation)

Chuyển văn bản từ ngôn ngữ này sang ngôn ngữ khác mà vẫn giữ nghĩa.

Ví dụ: I love studying NLP. → Tôi thích học NLP.

10 Trả lời câu hỏi (Question Answering)

Tìm câu trả lời ngắn gọn cho câu hỏi dựa trên ngữ cảnh hoặc tri thức đã cho.

Ví dụ: Đại học Thủy lợi nằm ở Hà Nội. (Ngữ cảnh)

Hỏi: Đại học Thủy lợi ở đâu? → **Hà Nội**

11 Tóm tắt văn bản (Text Summarization)

Rút gọn nội dung dài thành bản tóm tắt ngắn nhưng vẫn giữ ý chính.

Ví dụ:

Văn bản gốc

NLP là lĩnh vực nghiên cứu...
Nó có nhiều ứng dụng trong đời sống như dịch máy, trả lời câu hỏi, tóm tắt văn bản...
NLP đang phát triển rất nhanh...

→ **Tóm tắt**

NLP giúp máy hiểu và tạo ngôn ngữ của con người.

12 Mô hình ngôn ngữ / Sinh văn bản (Language Modeling / Text Generation)

Dự đoán từ tiếp theo hoặc sinh ra đoạn văn mới mạch lạc từ một lời nhắc.

Ví dụ: AI đang thay đổi... → cách con người học tập, làm việc và giao tiếp.

Từ tiếp theo (có thể là): cách | thế giới | cuộc sống | ...

NLP truyền thống

Các phương pháp NLP truyền thống trước Deep Learning

Những hướng tiếp cận chính trước kỷ nguyên neural sâu

1 Luật và ký hiệu (Rule-based / Symbolic)

Xây dựng hệ thống dựa trên các luật do con người thiết kế, bao gồm biểu thức chính quy, mẫu hữu hạn trạng thái, luật ngữ pháp và các quy tắc dạng if – then.

Ví dụ

- **Trích xuất email**
  Email: contact@hust.edu.vn
- **Trích xuất số điện thoại**
  SĐT: 0912-345-678
- **Sửa lỗi chính tả**
  Tôi thik học NLP → Tôi thích học NLP
- **Chatbot kịch bản**
  Nếu người dùng hỏi về học phí → trả lời theo kịch bản định sẵn

Các kỹ thuật tiêu biểu

- Regular expressions (regex)
- Grammar rules
- Finite-state automata (FSA)
- Expert rules (if – then logic)

2 Từ điển & tri thức ngôn ngữ (Lexicon-based)

Sử dụng các từ điển, cơ sở tri thức ngôn ngữ như WordNet, thesaurus, từ điển cảm xúc và ontology để khai thác tri thức ngôn ngữ có sẵn.

Ví dụ

- **Phân tích cảm xúc (đếm từ tích cực/tiêu cực)**
  Câu: "Sản phẩm này rất tốt nhưng giá hơi cao."
  Tích cực: 2 Tiêu cực: 1 → Cảm xúc: tích cực
- **Tìm từ đồng nghĩa (WordNet)**
  good (adj) → {good, fine, nice, excellent, ...}
- **Tra cứu trong ontology**
  "Hà Nội" → là một "Thành phố" → thuộc "Việt Nam"

Các kỹ thuật tiêu biểu

- WordNet
- Sentiment lexicon
- Ontology / Knowledge base
- Từ điển song ngữ (Bilingual dictionary)

3 Mô hình thống kê xác suất

Mô hình hóa ngôn ngữ bằng các xác suất thống kê, ước lượng từ dữ liệu. Bao gồm mô hình ngôn ngữ n-gram, Naive Bayes, HMM, các mô hình dịch thống kê (IBM Models) và ngữ pháp phi ngữ cảnh xác suất (PCFG).

Ví dụ

- **Mô hình ngôn ngữ n-gram**
  $P(w_t \mid w_{t-1}, w_{t-2})$
  P(học | tôi, thích) = 0.21
- **Gán nhãn từ loại (POS tagging)**
  Câu: Tôi thích học NLP .
  → Tôi/PRP thích/VBP học/VB NLP/NNP ./.
- **Dịch máy thống kê**
  $P(f \mid e) = \prod_i P(f_i \mid e_i)$
  I like cats → Tôi thích mèo

Các kỹ thuật tiêu biểu

- n-gram language model
- Hidden Markov Model (HMM)
- Naive Bayes
- IBM Models (thống kê dịch máy)
- Probabilistic CFG (PCFG)

4 Học máy truyền thống + đặc trưng thủ công

Biểu diễn văn bản bằng các đặc trưng do con người thiết kế, sau đó sử dụng các thuật toán học máy truyền thống để huấn luyện mô hình.

Quy trình tổng quát

Văn bản đầu vào → Trích xuất đặc trưng thủ công → Mô hình học máy → Kết quả dự đoán

- Văn bản đầu vào: "Email này quá hay!"
- Trích xuất đặc trưng thủ công: Bag-of-Words, TF-IDF, n-grams, Tiền tố / Hậu tố, POS tags, Các đặc trưng khác
- Mô hình học máy: SVM, Logistic Regression, Maximum Entropy (MaxEnt), Conditional Random Fields (CRF)
- Kết quả dự đoán: spam / không spam, tích cực / tiêu cực, nhãn thực thể (NER), ...

5 Không gian vector & mô hình chủ đề

Biểu diễn văn bản và từ bằng các vector trong không gian nhiều chiều, sử dụng độ tương đồng (như cosine similarity), các phương pháp giảm chiều (LSA/LSI) và mô hình chủ đề (LDA).

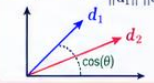
Ví dụ

Ma trận TF-IDF (rút gọn)

từ	D1	D2	D3
học	0.8	0.1	0.0
máy	0.2	0.7	0.1
dữ liệu	0.1	0.4	0.9
...	...	...	...

Độ tương đồng cosine

$$\text{sim}(d_1, d_2) = \frac{d_1 \cdot d_2}{\|d_1\| \, \|d_2\|}$$



Ứng dụng

- Tìm kiếm tài liệu (document search)
- Phân cụm tài liệu tương tự (document clustering)
- Khám phá chủ đề với LDA (ví dụ: "thể thao", "kinh tế", "giáo dục", ...)

Các kỹ thuật tiêu biểu

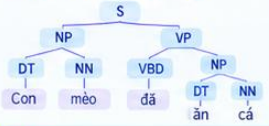
- Vector space model
- TF-IDF
- Cosine similarity
- Latent Semantic Analysis (LSA / LSI)
- Latent Dirichlet Allocation (LDA)

6 Phân tích cú pháp & cấu trúc ngôn ngữ

Phân tích cấu trúc ngữ pháp của câu nhằm hiểu quan hệ giữa các từ. Bao gồm phân tích cú pháp theo cụm (constituency parsing), phụ thuộc (dependency parsing), tách cụm (chunking) và suy luận vai nghĩa (SRL).

Ví dụ

Cây cú pháp (Constituency parsing)



Phụ thuộc (Dependency parsing)



Ứng dụng

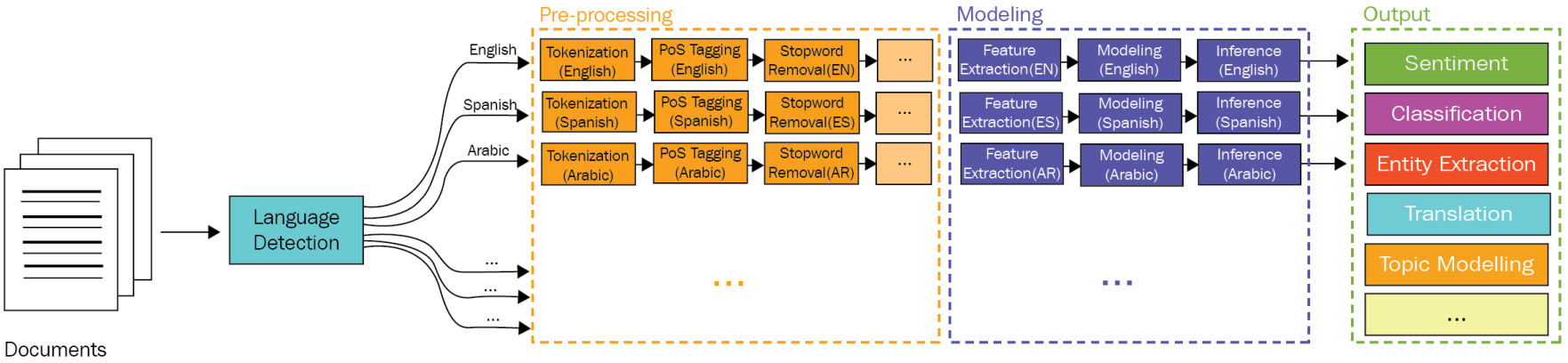
- Trích xuất thông tin (information extraction)
- Trả lời câu hỏi (question answering)
- Hiểu quan hệ và vai nghĩa giữa các thực thể

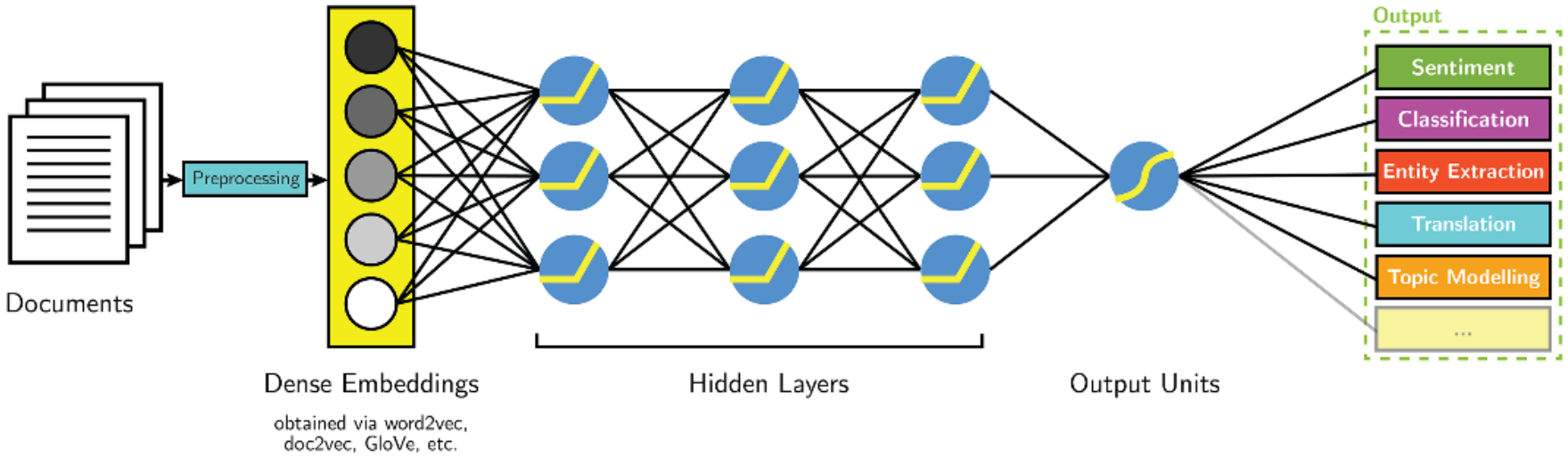
Các kỹ thuật tiêu biểu

- Constituency parsing
- Dependency parsing
- Chunking (NP, VP, ...)
- Semantic role labeling (SRL)
- Parse tree

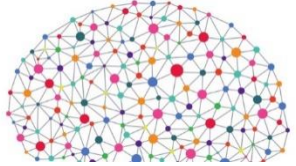
Đặc trưng chung	Phụ thuộc tri thức chuyên gia	Đặc trưng thủ công	Dễ diễn giải	Cần ít dữ liệu hơn deep learning	Khó nắm bắt ngữ cảnh dài và đa nghĩa

Classical NLP vs Deeplearning-based NLP



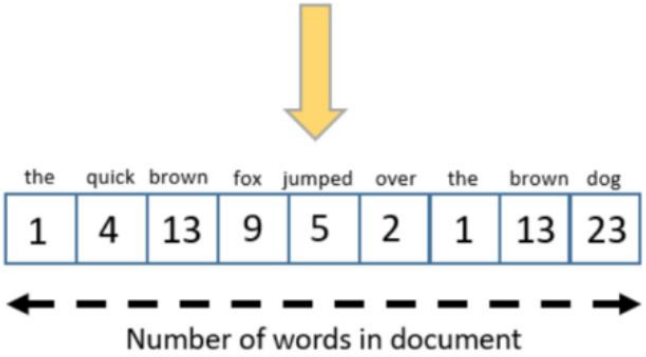


Mã hóa thông thường

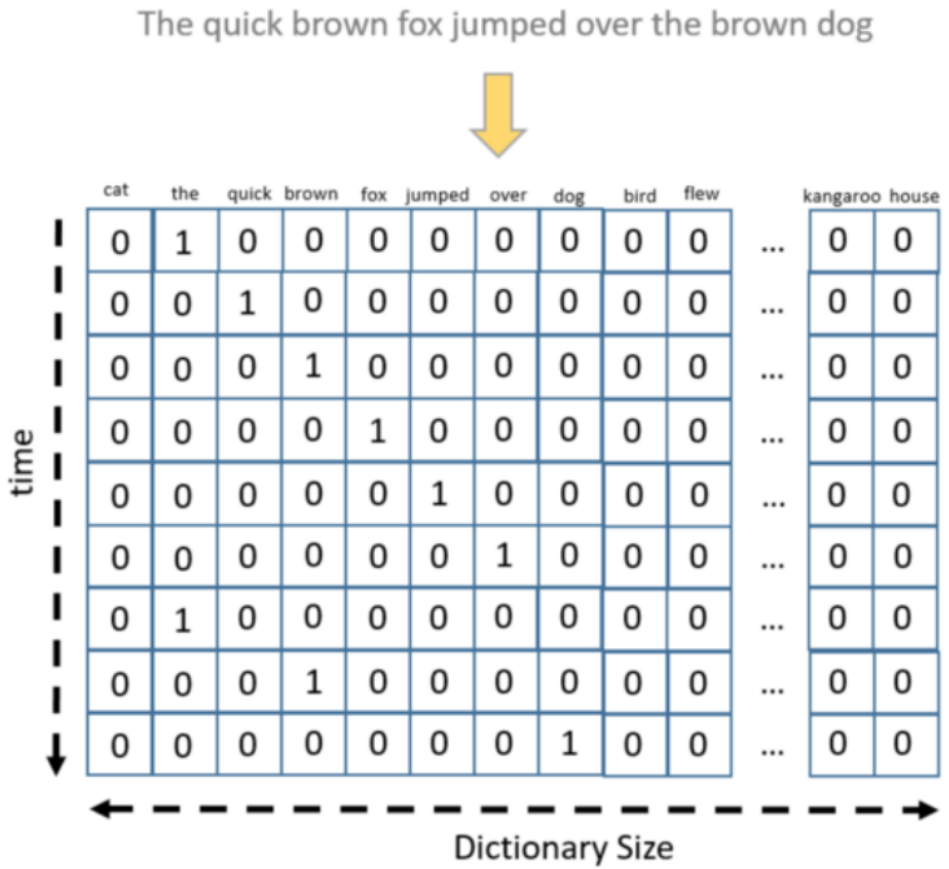


- Index based encoding:

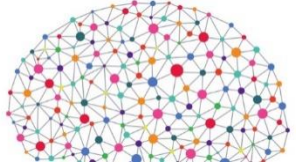
The quick brown fox jumped over the brown dog



- One-hot encoding:



Mã hóa theo thống kê



- Giải thiết rằng những từ gần gũi nhau thì có đặc trưng thống kê tương tự nhau (có cách sử dụng giống nhau)
- Count vector: đếm tần xuất của từ trong văn bản
- tf-idf vector: tần xuất của từ trong tập dữ liệu
- Co-occurrence matrix: đếm số lần cặp từ được sử dụng trong cùng ngữ cảnh (câu)

$$tf_i = \frac{n_i}{N_i}$$

$$idf_i = log_2 \frac{D}{d}$$

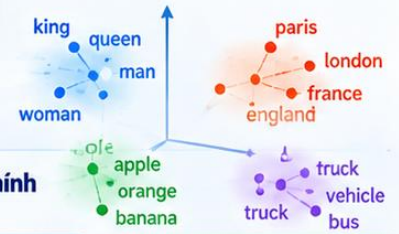
$$tfidf_i = tf_i \times idf_i$$

$$X = \begin{array}{c} \\ I \\ like \\ enjoy \\ deep \\ learning \\ NLP \\ flying \\ . \end{array} \begin{array}{c} \begin{array}{cccccccc} I & like & enjoy & deep & learning & NLP & flying & . \end{array} \\ \begin{bmatrix} 0 & 2 & 1 & 0 & 0 & 0 & 0 & 0 \\ 2 & 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{bmatrix} \end{array}$$

Word2Vec

Biểu diễn từ bằng vector – Nắm bắt ý nghĩa từ ngữ trong không gian số

> "You shall know a word by the company it keeps"
> — J. R. Firth (1957)
>
> Một từ được hiểu qua những từ xuất hiện cùng nó.



1 Bài toán là gì?

Học cách biểu diễn mỗi từ thành một vector số thực trong không gian liên tục sao cho:

- Các từ có ngữ nghĩa tương tự nằm gần nhau
- Giữ được quan hệ ngữ nghĩa (vi dụ: *king - man + woman ≈ queen*)
- Vector có thể từ dụng cho nhiều bài toán NLP khác (như phân loại, tìm kiếm, dịch máy, ...)

Ví dụ:

king - man + woman ≈ **queen**

paris - france + **italy** ≈ rome

2 Lịch sử ra đời

- **2001** **Bengio et al.** — Ý tưởng dùng mạng nơ-ron để học biểu diễn từ (NNLM)
- **2003** **Mikolov** (luận văn tiến sĩ) — Giới thiệu mô hình đơn giản, hiệu quả hơn (tiền thân của Word2Vec)
- **2013** **Mikolov et al. (Google)** — Công bố Word2Vec (CBOW & Skip-gram) Hiệu quả, huấn luyện nhanh trên dữ liệu lớn
- **2014** **Phổ biến mạnh mẽ** — Được sử dụng rộng rãi trong nhiều bài toán NLP và lĩnh vực khác

3 Ý tưởng chính

Dựa trên **giả thuyết ngữ cảnh**:
"Những từ xuất hiện trong ngữ cảnh tương tự thường có ý nghĩa tương tự".

"con mèo ngồi trên chiếc ghế"

mèo

con | ngồi | trên | chiếc | ghế

Học vector sao cho từ "mèo" gần với các từ thường xuất hiện cùng nó, và xa với các từ không liên quan.

4 Hai mô hình chính

CBOW (Continuous Bag-of-Words)

Dự đoán từ trung tâm dựa trên các từ ngữ cảnh xung quanh.

con, ngồi, trên, ghế → Average → mèo

- Nhanh, ổn định, hoạt động tốt với tập dữ liệu lớn.
- Phù hợp cho các từ phổ biến.

Skip-gram

Dự đoán các từ ngữ cánh xung quanh dựa trên từ trung tâm.

mèo → con, ngồi, trên, ghế

- Học tốt các từ hiếm.
- Thường cho kết quả tốt hơn trong nhiều bài toán.

5 Quy trình tạo ra Word2Vec

1. Dữ liệu văn bản

Văn bản thô (sách, báo, web, ...)

"Hôm nay trời đẹp .
Tôi đi học .
Học AI rất thú vị ..."

2. Tiền xử lý

- Tách câu, tách từ (tokenization)
- Làm sạch văn bản
- Xây dựng từ vựng (vocabulary)
- Chuyển thành chuỗi chỉ số (ID)

3. Tạo dữ liệu huấn luyện

Sinh các cặp huấn luyện từ cửa sổ ngữ cánh (window)

... con **mèo** ngồi ...
→ (context: con, ngồi) → **mèo**

... ngồi **trên** ghế ...
→ (context: ngồi, ghế) → **trên**

... (với Skip-gram thì đảo ngược)

4. Huấn luyện mô hình

Sử dụng mạng nơ-ron nông (1 hidden layer) với kỹ thuật:

- **Hierarchical Softmax** (HS)
  hoặc
- **Negative Sampling** (NS)
- Tối ưu bằng SGD

5. Kết quả

Mỗi từ được biểu diễn bởi một vector số thực d (thường 100–300 chiều).

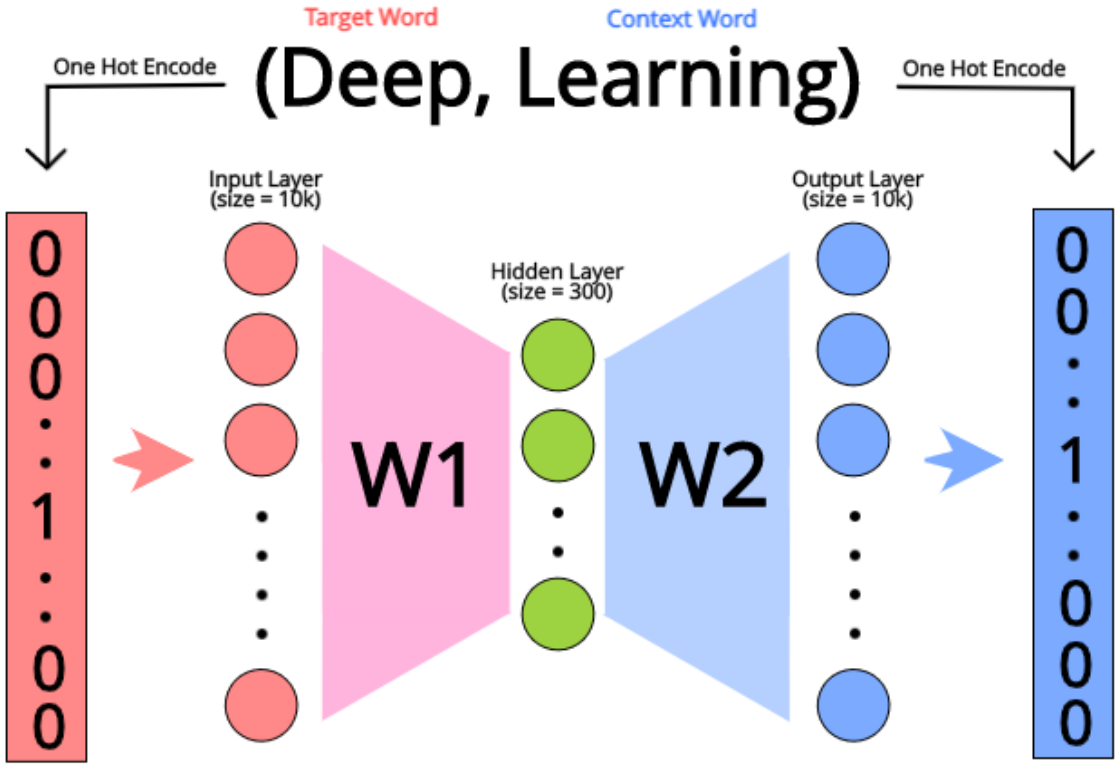
mèo	[0.12, -0.03, 0.55, ...]
chó	[0.10, -0.01, 0.50, ...]
xe	[-0.20, 0.30, -0.10, ...]
...	[...]

6 Ứng dụng

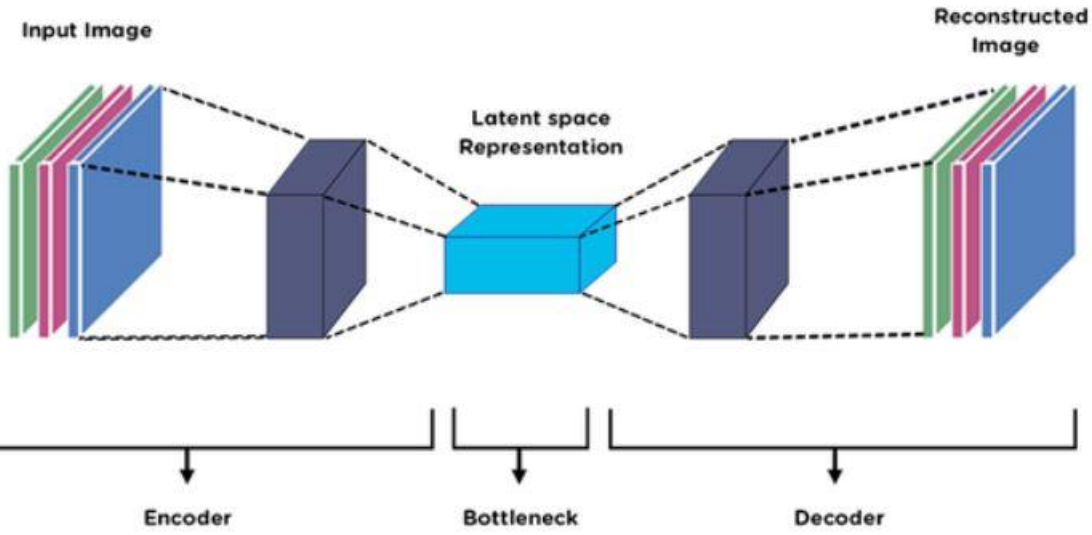
- Làm đặc trưng cho các mô hình NLP downstream (phân loại, NER, ...)
- Tìm kiếm và mở rộng truy vấn (thông tin, gợi ý)
- Phân tích **ngữ** nghĩa, phân cụm từ
- Phát hiện quan hệ tương đồng / dị biệt
- Là nền tảng cho các mô hình ngôn ngữ sau này (ELMo, BERT, ...)

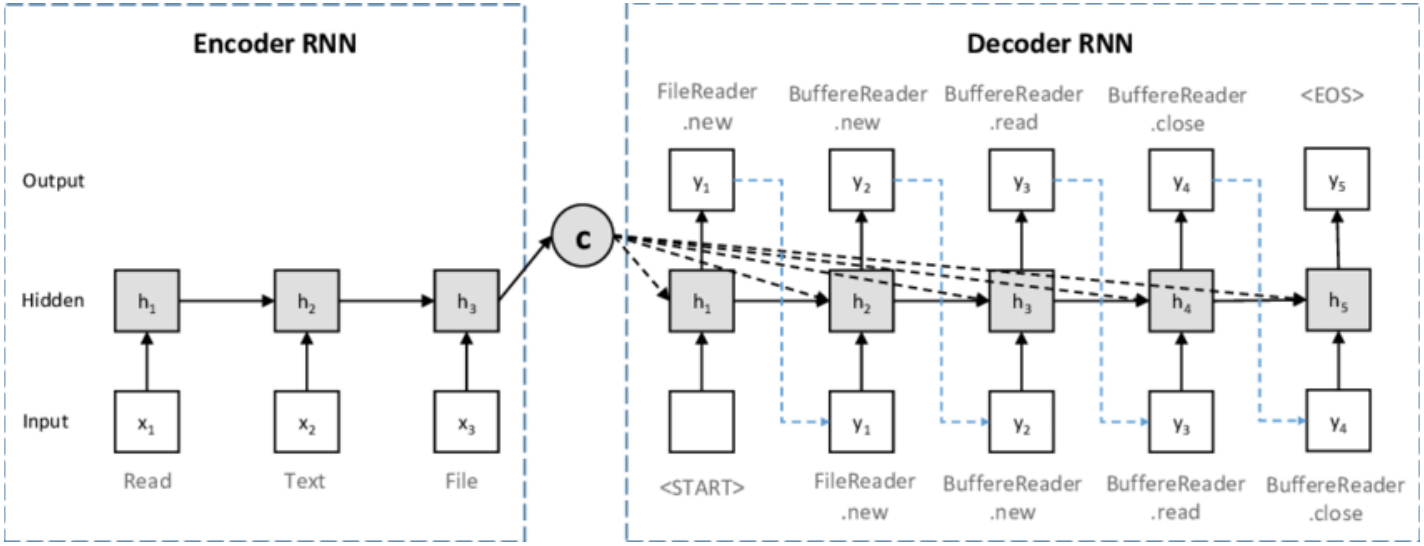
Nhúng từ

- Word2Vec:
  - Biến đổi từ không gian từ vựng sang không gian ngữ nghĩa
  - Biến đổi mỗi từ về một vector k chiều
  - Sử dụng mạng encoder nông

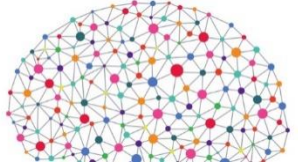


Encoder-decoder model

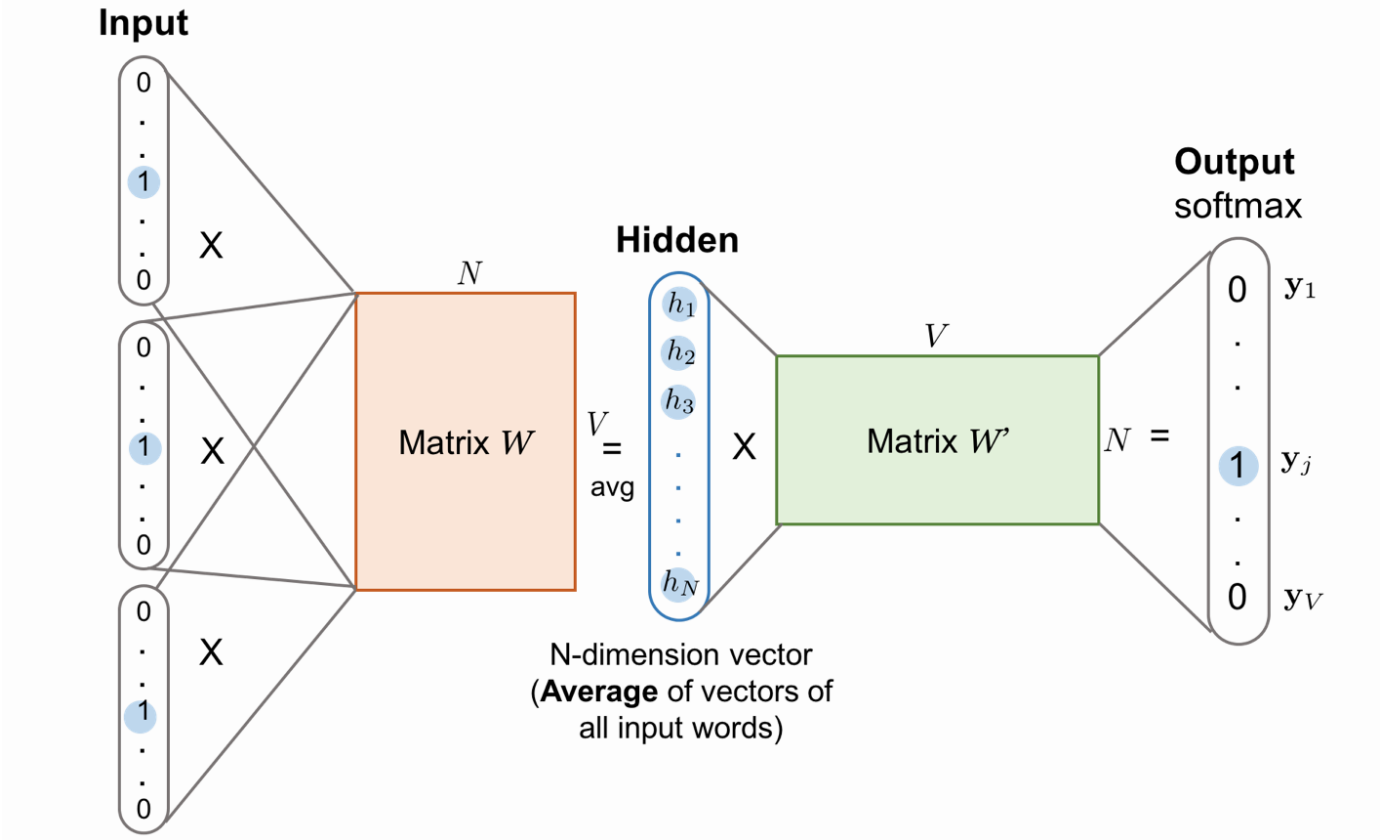




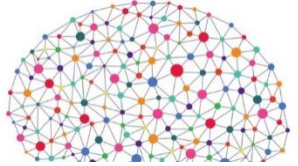
Continuous Bag-of-Words (CBOW)



- Cho context (các từ liên quan), đoán từ chính
- Train nhanh, độ chính xác cao hơn đối với những từ thông dụng



Continuous Bag-of-Words (CBOW)



CBOW trong Word2Vec

Dự đoán từ trung tâm từ các từ ngữ cảnh

1 CBOW là gì?

- CBOW (Continuous Bag of Words) là mô hình học từ vector bằng cách dự đoán từ trung tâm từ các từ ngữ cảnh xung quanh.
- Là một trong hai kiến trúc kinh điển của Word2Vec, được đề xuất bởi Mikolov và cộng sự (2013).

Các từ ngữ cảnh (đầu vào)

Tôi | thích | NLP | mỗi

→ CBOW →

học

Từ trung tâm (dự đoán)

2 Ý tưởng cốt lõi

- Những từ xuất hiện trong các ngữ cảnh tương tự thường có ý nghĩa tương tự và sẽ có vector gần nhau trong không gian số.
- CBOW coi ngữ cảnh như một tập từ (bag-of-words), không quá chú trọng đến thứ tự của các từ.

"Nếu hai từ thường xuất hiện trong những ngữ cảnh giống nhau → chúng có vector tương tự nhau."

Ví dụ cửa sổ ngữ cảnh (window size = 2)

Tôi | thích | **học** | NLP | mỗi | ngày

Ngữ cảnh trái (2 từ) — Ngữ cảnh phải (2 từ)

Đầu vào (ngữ cảnh): [Tôi, thích, NLP, mỗi]

Nhãn (từ trung tâm): học

Trong CBOW:

- Quan tâm đến các từ xung quanh, không quá chú trọng thứ tự.
- Cùng một ngữ cảnh (xuất hiện cùng nhau) → vector của các từ sẽ gần nhau.

3 Quy trình thực hiện CBOW

1. Dữ liệu văn bản

Corpus văn bản thô (sách, báo, web, ...)

"Tôi thích học NLP mỗi ngày ..."

2. Tiền xử lý

- Tách từ (tokenization)
- Làm sạch văn bản
- Xây dựng vocabulary
- Ánh xạ từ → ID

Ví dụ: "học" → 17

3. Tạo cặp huấn luyện

Từ cửa sổ ngữ cảnh, lấy các từ xung quanh làm đầu vào và từ trung tâm làm nhãn.

Ngữ cảnh: [Tôi, thích, NLP, mỗi]

Nhãn: học

4. One-hot encoding

"Tôi"

0 0 1 0 ... 0

Vector one-hot (kích thước |V|)

Làm tương tự cho các từ ngữ cảnh khác.

5. Tra cứu embedding

Dùng ma trận $W \in \mathbb{R}^{|V| \times d}$ để lấy vector cho mỗi từ.

Từ	ID	Vector embedding (d = 5)
Tôi	1	[0.2 -0.1 0.3 ...]
thích	5	[-0.4 0.7 0.1 ...]
NLP	9	[0.6 -0.2 0.5 ...]
mỗi	8	[-0.1 0.3 -0.4 ...]
...	...	... ...

6. Gộp vector

Lấy trung bình (hoặc tổng) các vector ngữ cảnh.

$$h = \frac{v_1 + v_2 + \ldots + v_n}{n}$$

Vector ngữ cảnh h (kích thước d)

7. Lớp tuyến tính

Nhân với ma trận $W' \in \mathbb{R}^{d \times |V|}$ để tạo score cho toàn bộ từ vựng.

$$u = W'h$$

u: vector score (kích thước |V|)

8. Softmax

Chuyển score thành xác suất.

$$p(w|context) = \frac{\exp(u_w)}{\sum_{i=1}^{|V|} \exp(u_i)}$$

P(w)

Tôi thích học ... ...

9. Tính loss

Dùng cross-entropy và cập nhật trọng số bằng backpropagation (SGD).

$$L = -\log p(w_{center} \mid context)$$

Cập nhật W, W' qua SGD

10. Kết quả cuối

Học được ma trận embedding **W** (vector cho mỗi từ).

Các vector này có thể dùng lại cho nhiều bài toán NLP khác.

4 Ví dụ minh họa từng bước

Câu: "Tôi thích học NLP mỗi ngày" | Window size = 2

Ví dụ 1: Dự đoán từ "học"

Ngữ cảnh: [Tôi, thích, NLP, mỗi] — Nhãn: học

One-hot (kích thước |V|)

Tôi	0	0	1	...	0
thích	0	1	0	...	0
NLP	0	0	0	...	1
mỗi	0	0	1	...	0

Tra cứu embedding (d = 3, ví dụ)

Tôi	[0.2	0.1	-0.3]
thích	[-0.1	0.4	0.2]
NLP	[0.5	-0.2	0.3]
mỗi	[-0.3	0.1	0.4]

Lấy trung bình

$$h = \frac{v_1 + v_2 + v_3 + v_4}{4}$$

[0.08 0.10 0.15]

(d = 3)

Softmax

P(Tôi)	0.01
P(thích)	0.02
P(học)	0.78
P(NLP)	0.10
...	...

Ví dụ 2: Dự đoán từ "NLP"

Ngữ cảnh: (thích, học, mỗi, ngày)

Nhãn: NLP

Tương tự các bước: one-hot → embedding → trung bình → softmax → dự đoán "NLP"

5 Đầu vào, đầu ra và tham số quan trọng

Đầu vào	Các từ ngữ cảnh (trong cửa sổ)
Đầu ra	Từ trung tâm (dự đoán)
Kích thước từ vựng |V|	Số lượng từ duy nhất (ví dụ: 10.000 – 1.000.000)
Kích thước embedding d	Số chiều vector (ví dụ: 100, 200, 300)
Window size	Số từ ngữ cảnh hai bên (ví dụ: 2, 5)
Learning rate	Tốc độ học (ví dụ: 0.001, 0.01)
Số epoch	Số lần lặp trên toàn bộ dữ liệu (ví dụ: 5 – 20)

Mẹo huấn luyện nhanh hơn

- **Negative Sampling:** chỉ phân biệt từ thật và một số từ nhiễu.
- **Hierarchical Softmax:** dùng cây nhị phân thay cho softmax đầy đủ.

6 Ưu điểm, hạn chế và ứng dụng

Ưu điểm

- Nhanh, đơn giản, dễ huấn luyện.
- Học tốt vector cho các từ phổ biến.
- Tạo embedding chất lượng, hữu ích cho nhiều bài toán NLP khác.

Hạn chế

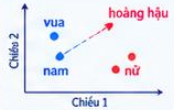
- Bỏ qua nhiều thông tin về trật tự từ.
- Chỉ nhìn ngữ cảnh hẹp (của số cố định).
- Mỗi từ chỉ có một vector tĩnh (không đa nghĩa).

Ứng dụng

- Biểu diễn từ trong không gian số.
- Tìm từ tương tự, suy luận quan hệ từ.
- Phân loại văn bản, clustering.
- Làm đầu vào cho các mô hình NLP khác (LSTM, BERT, ...).

Ví dụ trong không gian vector

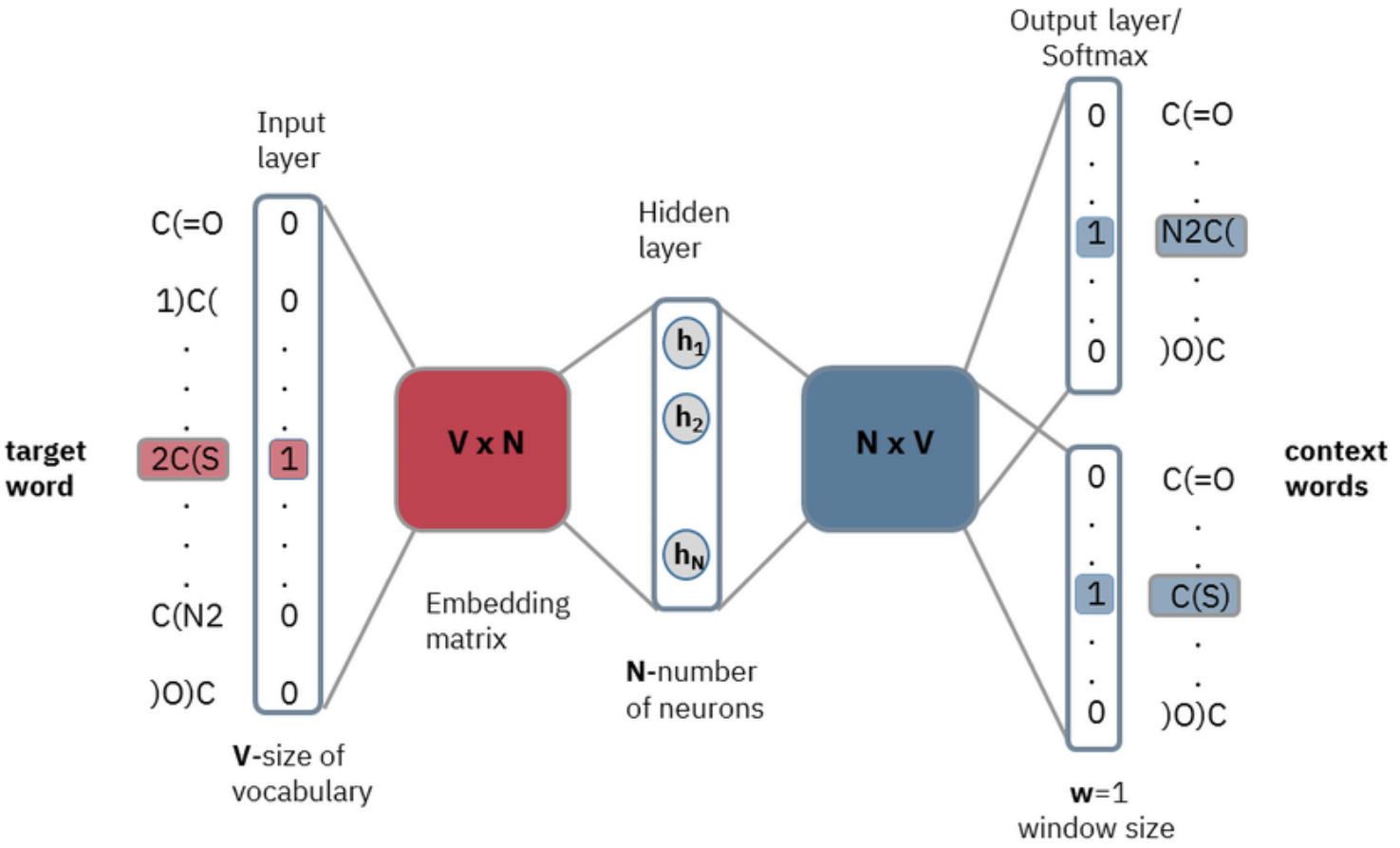
vua – nam + nữ ≈ hoàng hậu



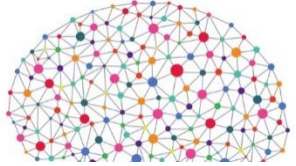
Các từ có ý nghĩa tương tự sẽ có vector gần nhau trong không gian số.

Skip-gram

- Cho từ chính, đoán các từ trong ngữ cảnh
- Tốt hơn CBOW với từ hiếm hoặc dữ liệu ít, training chậm



Skip-gram



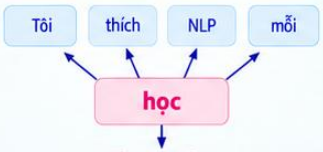
Skip-gram trong Word2Vec

Dự đoán các từ ngữ cảnh từ từ trung tâm

1 Skip-gram là gì?

- **Skip-gram (Skip-gram of Words)** là mô hình học từ vector bằng cách dự đoán các từ ngữ cảnh xung quanh từ một từ trung tâm.
- Là một trong hai kiến trúc kinh điển của **Word2Vec**, được đề xuất bởi **Mikolov** và cộng sự (2013).

Từ trung tâm dự đoán các từ ngữ cảnh (window size = 2)



2 Ý tưởng cốt lõi

- Giả thuyết phân bố (distributional hypothesis): Những từ xuất hiện trong các ngữ cảnh tương tự thường có nghĩa tương tự.
- Skip-gram học các vector từ sao cho những từ xuất hiện trong ngữ cảnh tương tự sẽ có vector gần nhau trong không gian số.

"Nếu hai từ thường xuất hiện trong những ngữ cảnh giống nhau → chúng có vector tương tự nhau."

Ví dụ cửa sổ ngữ cảnh (window size = 2)

Tôi	thích	**học**	NLP	mỗi	ngày
-2	-1	0	+1	+2	

Từ trung tâm: **học**

Các từ ngữ cảnh: **Tôi, thích, NLP, mỗi**

Trong Skip-gram:

- Đầu vào là một từ trung tâm.
- Đầu ra là nhiều từ ngữ cảnh lân cận.
- Thường học tốt hơn các từ hiếm (rare words) so với CBOW.

3 Quy trình thực hiện Skip-gram

1. Dữ liệu văn bản

Corpus văn bản thô (sách, báo, web, ...)

"Tôi thích học NLP mỗi ngày ..."

2. Tiền xử lý

- Tách từ (tokenization)
- Làm sạch văn bản
- Xây dựng vocabulary
- Ánh xạ từ → ID

Ví dụ: "học" → 17

3. Tạo cặp huấn luyện

Từ một từ trung tâm, tạo nhiều cặp (trung tâm, ngữ cảnh) với các từ trong cửa sổ.

Ví dụ (window = 2):
(học → Tôi)
(học → thích)
(học → NLP)
(học → mỗi)

4. One-hot encoding

"học"

0 0 0 1 0 ... 0

Vector one-hot (kích thước |V|)

Chỉ từ trung tâm được mã hóa, mỗi một lần huấn luyện.

5. Tra cứu embedding

Tra cứu ma trận W để lấy vector của từ trung tâm.

$W \in \mathbb{R}^{|V| \times d}$

Từ	ID	Vector embedding (d = 5)		
Tôi	1	0.2	0.1	...
thích	5	0.4	0.0	...
học	17	-0.3	0.8	...
NLP	9	0.1	-0.2	...
mỗi	21	0.5	0.1	...
...	...	...	...	...

6. Lớp tuyến tính đầu ra

Nhân với ma trận W' để tính điểm cho toàn bộ từ vựng.

$$u = h\,W'$$

$u \in \mathbb{R}^{|V|}$

$W' \in \mathbb{R}^{d \times |V|}$

Mỗi từ trong từ vựng đều có một score (kích thước |V|)

7. Softmax

Chuyển score thành xác suất phân phối trên toàn bộ từ vựng.

$$P(w|c) = \frac{\exp(u_w)}{\sum_{i=1}^{|V|} \exp(u_i)}$$

P(w)

Từ trong từ vựng V

8. Tính loss

Dùng hàm mất mát cross-entropy.

$$L = -\log P(w_{\text{ngữ cảnh}} \mid w_{\text{trung tâm}})$$

So sánh phân phối dự đoán với từ ngữ cảnh thật (one-hot target).

9. Cập nhật trọng số

Dùng backpropagation và SGD (hoặc thuật toán tối ưu khác) để cập nhật ma trận W và W'.

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

$$W' \leftarrow W' - \eta \frac{\partial L}{\partial W'}$$

η: learning rate

10. Kết quả cuối

Thu được ma trận embedding W (kích thước $|V| \times d$)

Các vector từ có thể dùng lại cho nhiều bài toán NLP khác.

4 Ví dụ minh họa từng bước

Câu: "Tôi thích học NLP mỗi ngày" | Window size = 2

Một từ trung tâm có thể tạo ra nhiều mục tiêu huấn luyện với các từ ngữ cảnh khác nhau.

Ví dụ 1: Dự đoán từ "thích" từ trung tâm "học"

1. One-hot (từ "học"): 0 0 0 1 0 ... (kích thước |V|)
2. Tra cứu embedding: $h = W^T x$; [-0.3 0.8 0.1 ...] (d = 3)
3. Softmax $P(w|\text{"học"})$:

Từ	Xác suất
thích	**0.62**
NLP	0.08
Tôi	0.15
mỗi	0.10
...	...

Ví dụ 2: Dự đoán từ "NLP" từ trung tâm "học"

1. One-hot (từ "học"): 0 0 0 1 0 ... (kích thước |V|)
2. Tra cứu embedding: $h = W^T x$; [-0.3 0.8 0.1 ...] (d = 3)
3. Softmax $P(w|\text{"học"})$:

Từ	Xác suất
NLP	**0.58**
thích	0.12
Tôi	0.14
mỗi	0.09
...	...

5 Đầu vào, đầu ra và tham số quan trọng

Thành phần	Mô tả
Đầu vào	Một từ trung tâm (center word)
Đầu ra	Các từ ngữ cảnh (nhiều từ trong cửa sổ)
Kích thước từ vựng $\|V\|$	Số lượng từ trong toàn bộ corpus (thường 10.000 – 1.000.000)
Kích thước embedding d	Số chiều vector từ (thường 50, 100, 200, 300)
Window size	Số từ ngữ cảnh hai bên (ví dụ: 2, 5)
Learning rate	Tốc độ học (ví dụ: 0.001, 0.01)
Số epoch	Số lần lặp trên toàn bộ dữ liệu (ví dụ: 5 – 20)

Có thể dùng các phương pháp huấn luyện nhanh hơn:

- **Negative Sampling:** chỉ phân biệt từ thật và một số từ nhiễu.
- **Hierarchical Softmax:** sử dụng cây nhị phân Huffman.

6 Ưu điểm, hạn chế và ứng dụng

Ưu điểm

- Học tốt cho các từ hiếm (rare words).
- Biểu diễn ngữ nghĩa tốt, vector chất lượng cao.
- Hữu ích cho nhiều tác vụ NLP khác nhau.

Hạn chế

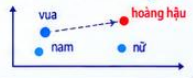
- Chậm hơn CBOW (vì phải dự đoán nhiều từ ngữ cảnh).
- Bỏ qua trật tự từ (word order).
- Chỉ dựa vào ngữ cảnh cục bộ (local context).

Ứng dụng

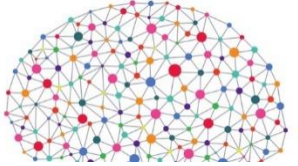
- Đo độ tương đồng giữa các từ.
- Tìm từ liên quan (nearest neighbors).
- Phân loại văn bản, clustering.
- Làm embedding đầu vào cho các mô hình NLP khác (BERT, RNN, CNN, ...).

Ví dụ trong không gian vector

vua - nam + nữ ≈ hoàng hậu



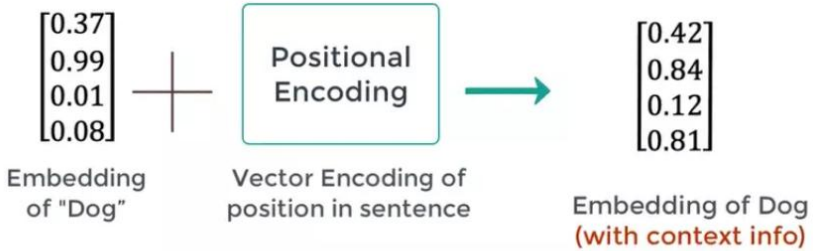
Các kĩ thuật hiện đại



- FastText:
  - Mô hình khá giống skip-gram
  - Dữ liệu đầu vào không phải là từ mà là một đoạn n chữ
  - Tốt với dữ liệu hiếm, teencode, viết sai chính tả,…
- BERT
  - Autoencoding, không còn là mạng neural nông
  - Che đi một số từ (15% độ dài), training mô hình đoán từ bị che
- XLNet
  - Autoregressive, không còn là mạng neural nông
  - Sử dụng dữ liệu đầu vào bị xáo trộn (permutation operation) thay vì che
  - Rất phù hợp với những từ ghép, tên riêng

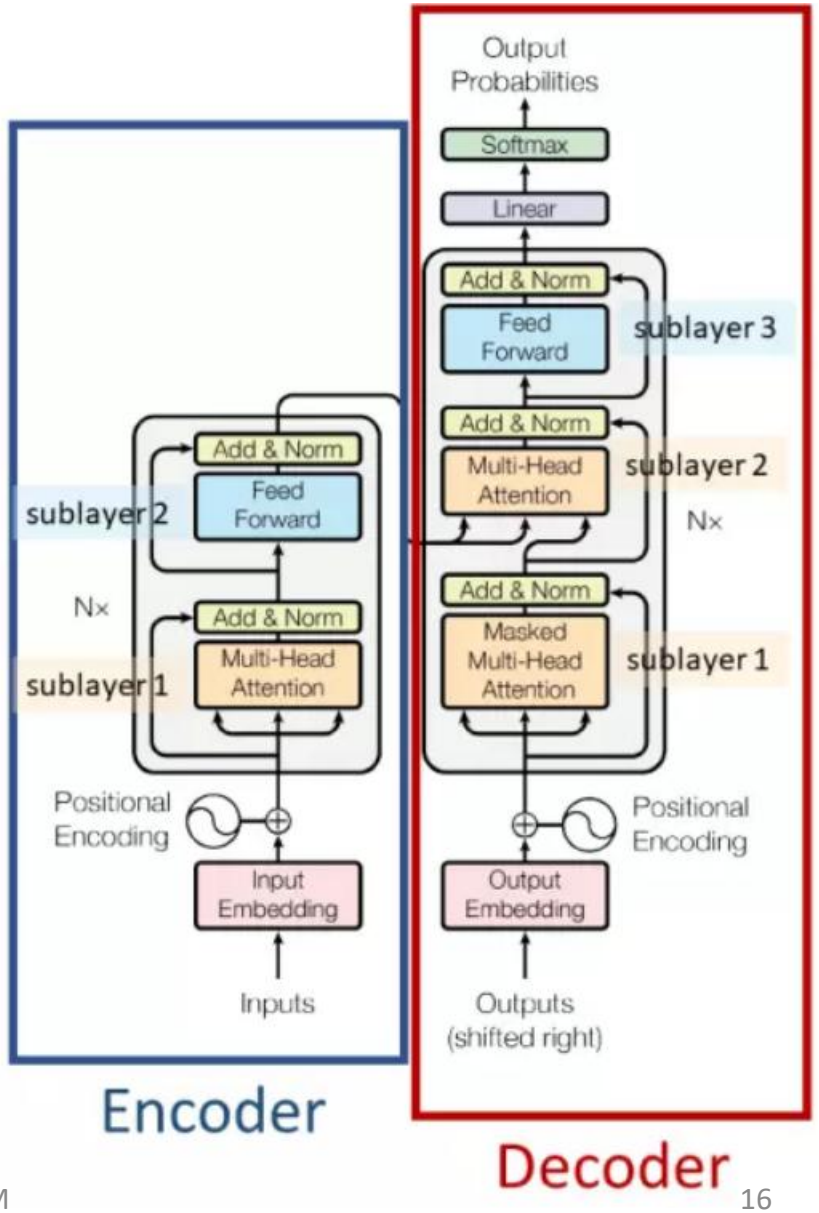
Mô hình transformer

- Cơ chế encoder-decoder
- Input embedding
- Mã hóa vị trí

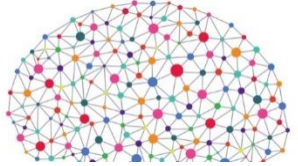


$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{model}})$$

- Cơ chế multi-head attention
- Masked multi-head attention
- Các khối residual



TextCNN

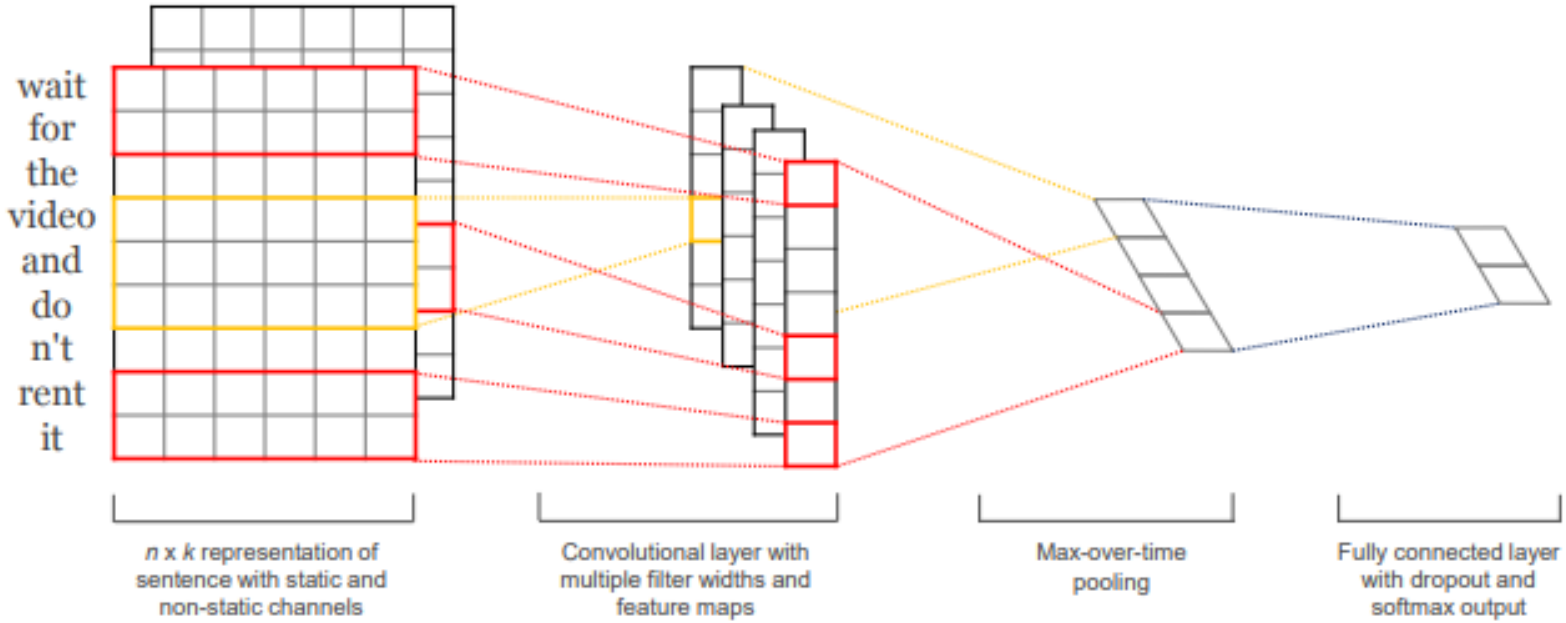


- Một cố gắng sử dụng CNN cho dữ liệu text
- Có kết quả khá tốt đối với các bộ dataset đầu ra phân lớp
- Không bị ràng buộc bởi phương pháp nhúng từ cố định
- Kiến trúc đơn giản, huấn luyện nhanh, chạy nhanh

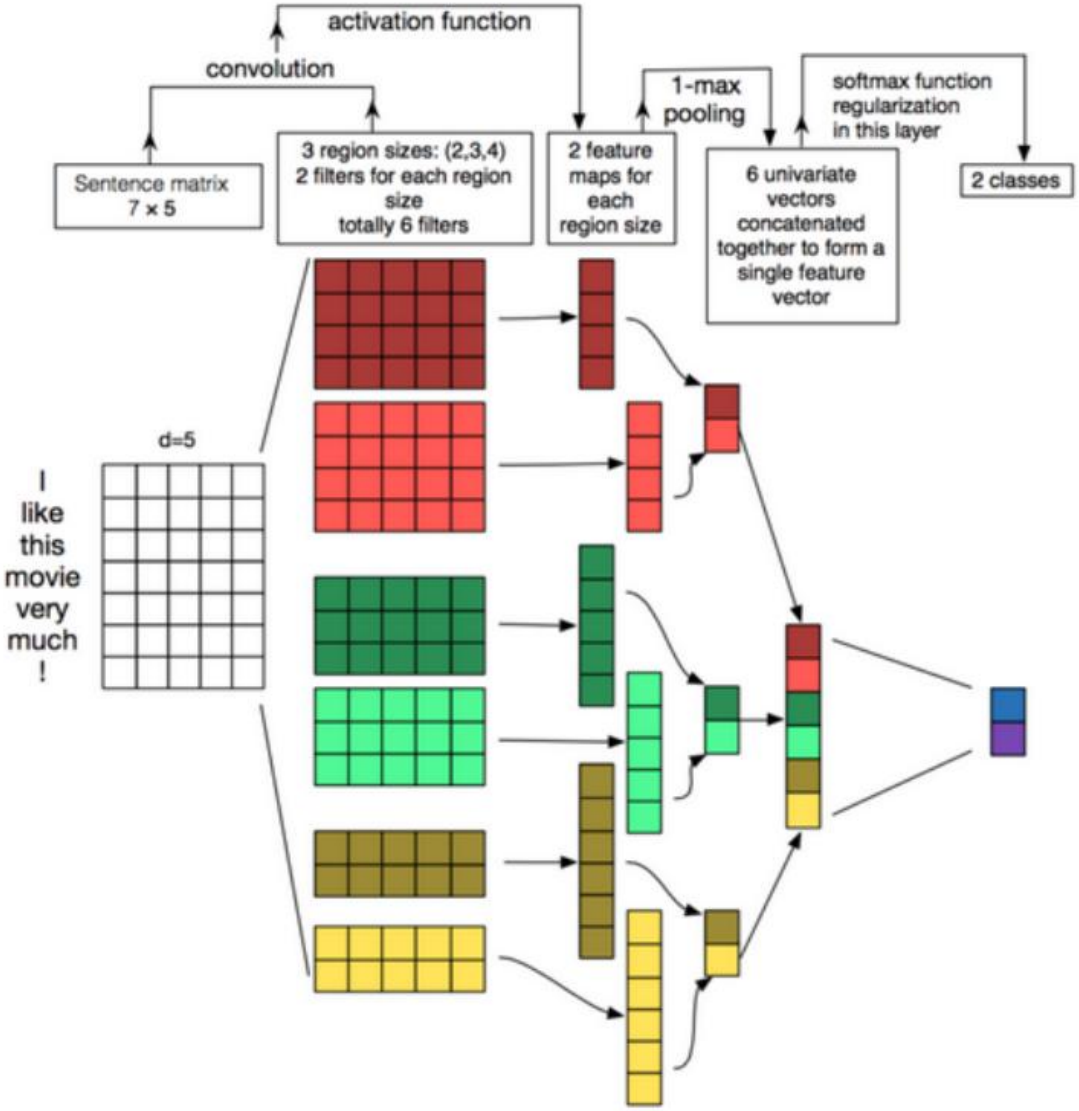
Model	MR	SST-1	SST-2	Subj	TREC	CR	MPQA
CNN-rand	76.1	45.0	82.7	89.6	91.2	79.8	83.4
CNN-static	81.0	45.5	86.8	93.0	92.8	84.7	**89.6**
CNN-non-static	**81.5**	48.0	87.2	93.4	93.6	84.3	89.5
CNN-multichannel	81.1	47.4	**88.1**	93.2	92.2	**85.0**	89.4
RAE (Socher et al., 2011)	77.7	43.2	82.4	–	–	–	86.4
MV-RNN (Socher et al., 2012)	79.0	44.4	82.9	–	–	–	–
RNTN (Socher et al., 2013)	–	45.7	85.4	–	–	–	–
DCNN (Kalchbrenner et al., 2014)	–	48.5	86.8	–	93.0	–	–
Paragraph-Vec (Le and Mikolov, 2014)	–	**48.7**	87.8	–	–	–	–
CCAE (Hermann and Blunsom, 2013)	77.8	–	–	–	–	–	87.2
Sent-Parser (Dong et al., 2014)	79.5	–	–	–	–	–	86.3
NBSVM (Wang and Manning, 2012)	79.4	–	–	93.2	–	81.8	86.3
MNB (Wang and Manning, 2012)	79.0	–	–	**93.6**	–	80.0	86.3
G-Dropout (Wang and Manning, 2013)	79.0	–	–	93.4	–	82.1	86.1
F-Dropout (Wang and Manning, 2013)	79.1	–	–	**93.6**	–	81.9	86.3
Tree-CRF (Nakagawa et al., 2010)	77.3	–	–	–	–	81.4	86.1
CRF-PR (Yang and Cardie, 2014)	–	–	–	–	–	82.7	–
SVM_S (Silva et al., 2011)	–	–	–	–	**95.0**	–	–

TextCNN

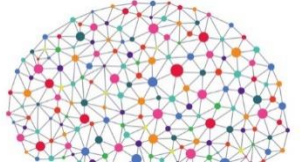
- Dữ liệu đầu vào là vector nhúng từ 300 chiều (CBOW)
- 300 bộ lọc tích chập gồm 3 loại kernel size 3, 4 hoặc 5
- Khối max-pooling
- Đầu ra phân lớp softmax



TextCNN


activation function
convolution
1-max pooling
softmax function regularization in this layer
Sentence matrix 7 × 5
3 region sizes: (2,3,4) 2 filters for each region size totally 6 filters
2 feature maps for each region size
6 univariate vectors concatenated together to form a single feature vector
2 classes
d=5
I
like
this
movie
very
much
!

TextCNN



Model	Sentiment						News		Topic	NLI
	MR	SST-2	IMDB	Yelp.P	Yelp.F	Amz.F	20NG	AG	DBpedia	SNLI
NB [8]	-	81.80	-	-	-	-	-	-	-	-
SVM [42]	-	79.40	-	-	-	-	-	-	-	-
Tree-CRF [284]	77.30	-	-	-	-	-	-	-	-	-
RAE [60]	77.70	82.40	-	-	-	-	-	-	-	-
MV-RNN [62]	79.00	82.90	-	-	-	-	-	-	-	-
RNTN [64]	75.90	85.40	-	-	-	-	-	-	-	-
DCNN [5]		86.80	89.40	-	-	-	-	-	-	-
Paragraph-Vec [67]		87.80	92.58	-	-	-	-	-	-	-
TextCNN[17]	81.50	88.10	-	-	-	-	-	-	-	-
TextRCNN [72]	-	-	-	-	-	-	96.49	-	-	-
DAN [69]	-	86.30	89.40	-	-	-	-	-	-	-
Tree-LSTM [1]		88.00	-	-	-	-	-	-	-	-
CharCNN [93]	-	-	-	95.12	62.05	-	-	90.49	98.45	-
HAN [108]	-	-	49.40	-	-	63.60	-	-	-	-
SeqTextRCNN [7]	-	-	-	-	-	-	-	-	-	-
oh-2LSTMp [75]	-	-	94.10	97.10	67.61	-	86.68	93.43	99.16	-
LSTMN [112]	-	87.30	-	-	-	-	-	-	-	-
Multi-Task [79]	-	87.90	91.30	-	-	-	-	-	-	-
BLSTM-2DCNN [77]	82.30	89.50	-	-	-	-	**96.50**	-	-	-
TopicRNN [83]	-	-	93.72	-	-	-	-	-	-	-
DPCNN [98]	-	-	-	97.36	69.42	65.19	-	93.13	99.12	-
KPCNN [100]	83.25	-	-	-	-	-	-	88.36	-	-
RNN-Capsule [87]	**83.80**		-	-	-	-	-	-	-	-
ULMFiT [285]	-	-	95.40	97.84	71.02	-	-	94.99	99.20	-
LEAM[286]	76.95	-	-	95.31	64.09	-	81.91	92.45	99.02	-
TextCapsule [101]	82.30	86.80	-	-	-	-	-	92.60	-	-
TextGCN [155]	76.74	-	-	-	-	-	86.34	67.61	-	-
BERT-base [19]	-	93.50	95.63	98.08	70.58	61.60	-	-	-	91.00
BERT-large [19]	-	94.90	95.79	98.19	71.38	62.20	-	-	-	91.70
MT-DNN[287]	-	95.60	83.20	-	-	-	-	-	-	91.50
XLNet-Large [138]	-	96.80	**96.21**	**98.45**	**72.20**	**67.74**	-	-	-	-
XLNet [138]	-	**97.00**	-	-	-	-	-	**95.51**	**99.38**	-
RoBERTa [140]	-	96.40	-	-	-	-	-	-	-	**92.60**